

Medical Device Data Aggregation System

Architecture & decision report — an **async-first** backend that ingests heterogeneous medical-device measurements, validates them, stores them idempotently in PostgreSQL, and runs reliable background jobs (clinical alerts, risk scores) over a transactional outbox and a broker-backed worker queue.

FastAPI · async/await

async SQLAlchemy · asyncpg

PostgreSQL

PgBouncer pooling

Transactional outbox

RabbitMQ · aio-pika consumers

Prometheus + Grafana

Docker Compose

Single `docker compose up` starts the full stack.

1 · At a glance

The system exposes one ingestion endpoint for three medical device types, enforces physiological validation, stores measurements idempotently, serves time-windowed aggregations, and — after each stored measurement — runs background jobs that raise clinical alerts and compute risk scores. The headline guarantees are **idempotency under concurrency** and **no background job lost** on a worker crash, relay outage, or service restart.

Requirement (spec)	How it is met
Single ingestion endpoint, 3+ device types	One <code>POST /ingest</code> with a flat Pydantic discriminated union on <code>device_type</code> (§4)
Physiological validation → 422	Per-type Pydantic field bounds; out-of-range rejected with a clear 422 (§5)
Idempotent duplicates	<code>UNIQUE(device_id, timestamp) + ON CONFLICT DO NOTHING</code> → 201 new / 200 duplicate (§5)
Background alert on threshold crossing	Outbox event → relay → RabbitMQ → async consumer persists the alert (§3, §6)
Broker-backed queue, no job lost on crash/restart	RabbitMQ durable queues + manual ack (at-least-once) + dead-letter table + replay (§6)
Async end-to-end	FastAPI + async SQLAlchemy + asyncpg; async aio-pika consumers (§7)
Device auth	<code>POST /devices/register</code> issues an API key; <code>X-Device-Key</code> required; 401 / 403 (§5)
Full stack via docker compose	10 services, one <code>docker compose up</code> (Appendix)

2 · Architecture (container view)

Two paths share one database. The **synchronous ingest path** (Client → API → PgBouncer → PostgreSQL) writes the measurement and a transactional-outbox row in one atomic transaction. The **asynchronous path** (Outbox Relay → RabbitMQ → consumers) delivers the background work. The API and both workers reach PostgreSQL **through PgBouncer**; only `migrate` and the relay connect directly (the relay needs `LISTEN/NOTIFY`, which a transaction pooler cannot proxy).

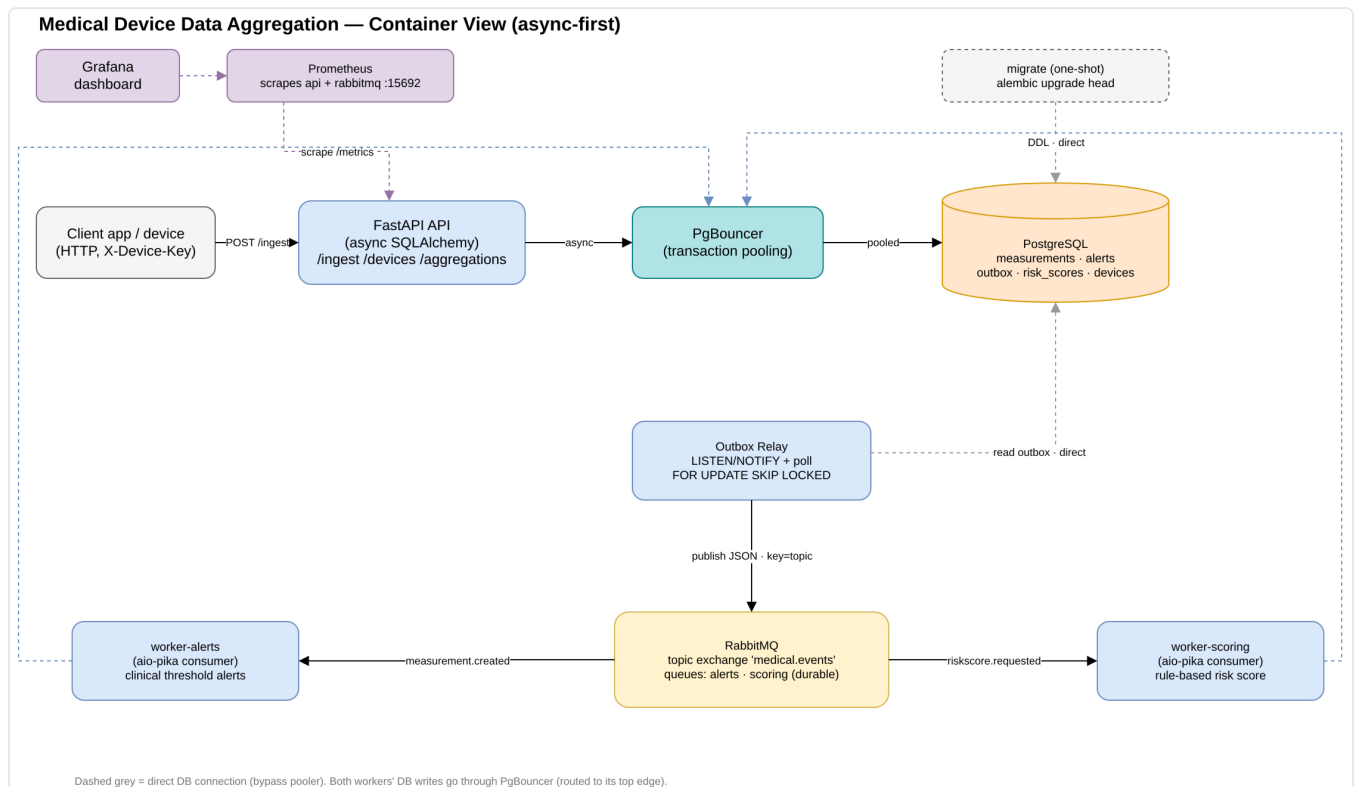


Figure 1 — Container view. Blue = app services; teal = pooler; orange = datastore/broker; purple = monitoring. Dashed = one-shot / scrape / direct-bypass edges.

3 • Decision narrative

First why this architecture and why these technologies, then each design decision as **what • why • trade-off**. The diagrams (Figures 1–3) are the visual companion to these decisions.

Why a message-driven pipeline (transactional outbox + relay)

The spec asks that, after a measurement is stored, a background task runs that must survive a worker crash or service restart. That requirement points away from doing the work inline (FastAPI `BackgroundTasks` die with the process) and toward an **event-driven, broker-backed** pipeline. Three properties make it the right shape here:

- **Fast, predictable ingest.** A request does one atomic DB write and returns; clinical alerting and risk scoring happen asynchronously, so a slow or expensive job never inflates ingest latency or fails the request.
- **Durability & decoupling.** A durable queue separates producer from consumer: consumers can crash, restart, or scale out independently, and the queue absorbs bursts as natural back-pressure (Figure 7).
- **No dual-write gap.** Writing the measurement and its event in one transaction means we never publish an event for data that didn't commit, nor lose an event for data that did. The relay then ships committed events to the broker at-least-once; idempotent consumers turn that into an exactly-once effect.

This is the standard reliable pattern for "do X, then dependably trigger Y", and it is what lets the system meet the spec's "no job lost" guarantee that an in-process mechanism cannot.

Technology choices

Technology	Why chosen	Considered instead
FastAPI (async)	Spec mandates async/await; high-performance ASGI; first-class Pydantic v2 integration.	— (required)
Pydantic v2	Discriminated unions + field validators give declarative physiological validation → type-specific 422 for free.	— (required)
PostgreSQL	Required by the spec — and it provides the exact primitives the design leans on: <code>UNIQUE + ON CONFLICT</code> (idempotency), <code>FOR UPDATE SKIP LOCKED</code> (relay concurrency), <code>LISTEN/NOTIFY</code> (low-latency relay trigger), ACID transactions (atomic outbox).	— (required)
async SQLAlchemy + asyncpg	Async SQLAlchemy is spec-mandated; asyncpg is the fastest async PostgreSQL driver; non-blocking DB I/O end to end.	— (required)
RabbitMQ	Durable queues + manual acks = the broker-backed, no-loss delivery the spec requires; per-queue isolation; operationally simple at this scale.	Kafka (heavier, log-based); Redis (weaker durability)
aio-pika	Async-native AMQP client that runs on the same event loop as the app (§ consumer decision).	pika (sync); Taskiq/Celery
PgBouncer	Bounds server connections so the stateless app tier scales without exhausting Postgres.	PgCat (incompatible with asyncpg — see below)
Prometheus + Grafana	De-facto metrics stack; the API already exposes <code>/metrics</code> .	—
Docker Compose	Spec deliverable: the full stack with a single <code>docker compose up</code> .	—

Transactional outbox (not direct publish to the broker)

Why: the measurement row and an `outbox` event commit in one DB transaction, so a crash between "DB committed" and "message sent" cannot lose the job — the event is durable iff the data is.

Trade-off: one extra moving part (the relay) and at-least-once (not exactly-once) delivery — absorbed by the idempotency below.

CDC-style relay over a poll loop: `LISTEN/NOTIFY` + 5 s fallback poll + `FOR UPDATE SKIP LOCKED`

Why: NOTIFY gives ~millisecond delivery; the poll is a safety net if a notification is missed; SKIP LOCKED lets several relay replicas share the backlog without ever processing a row twice (horizontal HA at the relay).

Trade-off: the relay re-publishes on retry, so consumers must be idempotent (they are).

Async aio-pika consumers, not a task framework (Taskiq/Celery)

Why: the relay publishes **plain JSON** to a RabbitMQ topic exchange (routing key = event topic). Consumers are ~80 readable lines that declare their queue, run the handler, and own delivery explicitly: manual `ack` only after success (at-least-once), bounded retry, then a dead-letter row. Transparent and auditable — valuable when "no lost alert" is the headline guarantee — and it keeps the broker contract a plain message, so a future CDC tool is a drop-in producer. Concurrency is the QoS prefetch window on one event loop (the same model a task framework would use).

Trade-off: we hand-roll the few queue features we use (ack/retry/DLQ) instead of inheriting them; for this scope that is a small, well-contained amount of code. RabbitMQ durable queues + manual acks still satisfy the spec's "broker-backed, no jobs lost" requirement (Taskiq/Celery were only examples).

Idempotency via DB constraints + ON CONFLICT DO NOTHING

Why: the database arbitrates duplicates under concurrency with no application locks —

`UNIQUE(device_id,timestamp)` (ingest), `UNIQUE(measurement_id,rule)` (alerts), `UNIQUE(request_id)` (risk scores).

At-least-once delivery therefore has an exactly-once effect.

Trade-off: an idempotency key must be designed per entity; correctness relies on Postgres semantics.

Connection pooling with PgBouncer (transaction mode)

Why: the stateless API and workers scale horizontally, but each replica keeps its own pool; past a point, replicas × pool exceeds Postgres `max_connections` and ingestion returns 500s. PgBouncer multiplexes many client connections onto a small bounded set of server connections, so the app tier scales without exhausting the database. `asyncpg` works behind a transaction pooler with its prepared-statement cache disabled (`statement_cache_size=0` , already set).

Trade-off & finding: **PgCat** (Rust, multi-threaded) was preferred for concurrency, but it is **incompatible with `asyncpg`** — `asyncpg`'s extended-protocol `Flush` message makes PgCat error `Unexpected code: H` in transaction, session, and prepared-statement-cache modes (libpq clients like `psql` work fine). PgBouncer is the proven `asyncpg` pooler, so we use it; PgCat would require switching the driver to libpq-based `psycopg3` .

Connection budget — the relay bypasses the pooler. The relay keeps **one** fixed direct connection (for `LISTEN/NOTIFY` + its batched outbox queries); it does not grow with load. So the budget is `PgBouncer pool + relay instances + reserves < max_connections` — here ≈ 40 (pool) + 1 (relay) ≈ 42 of 100, ample headroom. Connection exhaustion (and the 500s it would cause, surfacing on the API path through the pooler) only appears if relays are scaled heavily or the pool is sized near `max_connections` ; even then no data is lost — unrelayed events stay `acked=false` and drain when capacity returns.

Single ingestion endpoint = flat discriminated union on `device_type`

Why: Pydantic reads the discriminator first and applies only that type's validators, so a blood-pressure payload is never checked against heart-rate rules and errors are type-specific. Open for extension, closed for modification — a new device type is one model + one union arm, no `if device_type == ...` anywhere.

Trade-off: the union must be kept in sync with the device types.

Validation bounds vs. clinical alert thresholds are separate concerns

Why: out-of-range values are an input-integrity problem → reject with 422. Valid-but-concerning values (`heart_rate>150` , `spo2<90` , `systolic>180`) are a clinical-signal problem → store and raise a background alert. Conflating them was the main contradiction in the provided load test; we follow the spec.

Trade-off: two sets of numbers to maintain (both centralised in `src/config.py`).

Dead-letter table with replay (not a silent drop)

Why: a poison message that exhausts its retries must not vanish — for medical data a missed alert is unacceptable. Each failure upserts one `failed_jobs` row (cleared on a later success); `GET /admin/failed-jobs` lists them and

`POST /admin/failed-jobs/{id}/replay` re-emits the original outbox event, which flows back through the normal pipeline (idempotent).

Trade-off: a small operational surface (DB table + admin endpoints) — which is exactly what medical auditability wants.

Worker split by workload profile (alerts vs scoring)

Why: latency-sensitive alerts and heavier risk-scoring run on separate queues/containers so a slow scoring job never blocks alert delivery (no head-of-line blocking). Both scale independently by replica count.

Trade-off: two worker services instead of one.

4 · Data model

Seven tables. `patients` → `devices` → `measurements` is the ingestion spine; each measurement fans out to `alerts` and to its transactional `outbox` event, while `risk_scores` hangs off `patients`. `failed_jobs` is intentionally standalone (no FK) so the dead-letter record survives even when related rows are absent or being replayed. Idempotency is enforced by the `UNIQUE` constraints shown; composite indexes back the aggregation and history queries.

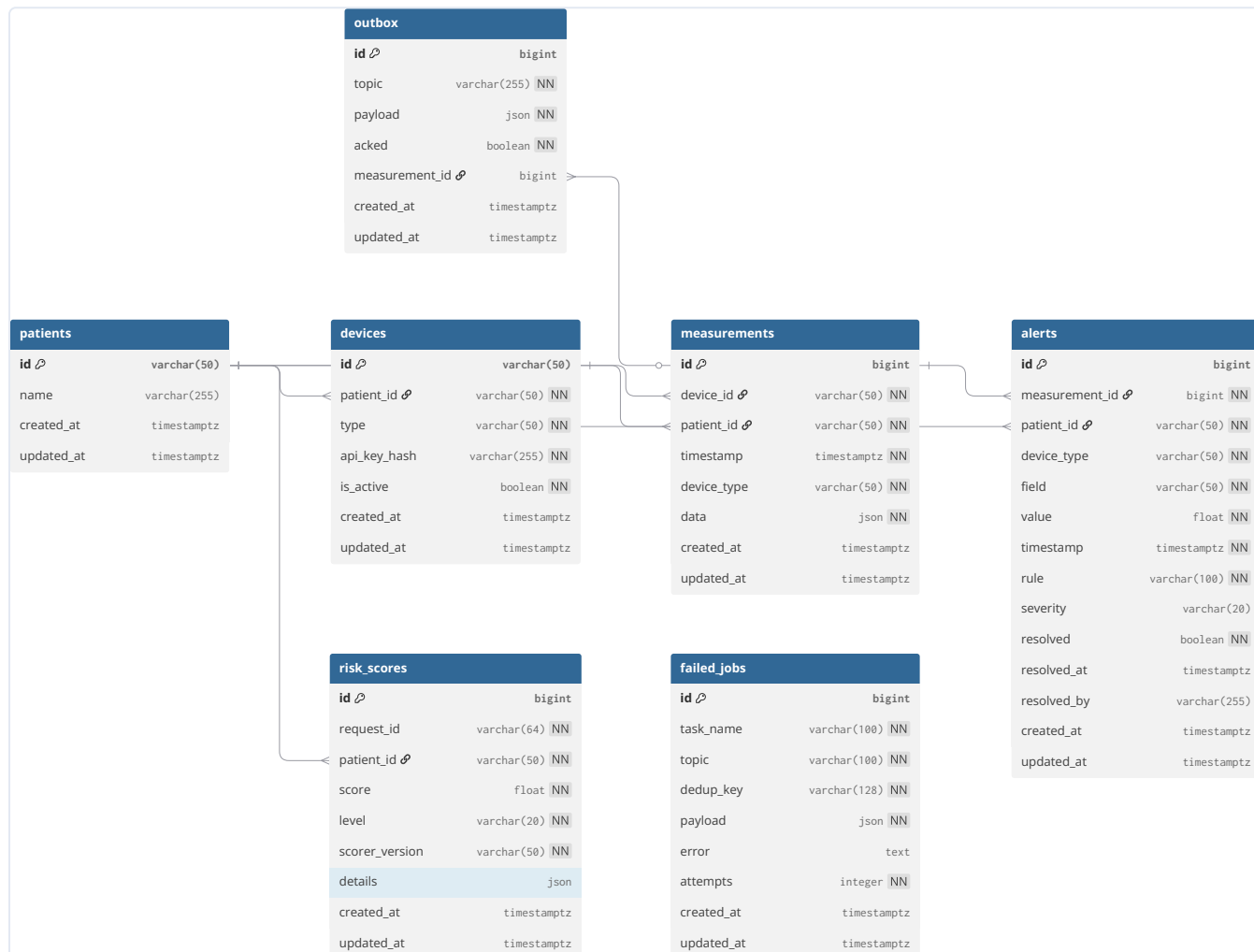


Figure 2 — Entity-relationship diagram (physical schema): seven tables; the `UNIQUE` constraints are the idempotency keys. The schema is unchanged from the baseline.

5 · Request handling — validation, idempotency, auth

Discriminated union & physiological validation

The body is `Annotated[Union[HeartRate, BloodPressure, PulseOximeter], Field(discriminator="device_type")]`. Each arm declares its own bounds — `heart_rate` ∈ [30,250], `systolic` ∈ [60,250], `diastolic` ∈ [40,150] with `systolic > diastolic`, `pulse` ∈ [30,250], `spo2` ∈ [50,100], `perfusion_index` > 0. A violation, a missing/unknown `device_type`, or fields that don't match the resolved type are rejected with **422**.

Idempotency under concurrent duplicates

`INSERT ... ON CONFLICT (device_id, timestamp) DO NOTHING`; the handler inspects `rowcount`: 1 → **201** (new; one outbox event emitted), 0 → **200** (duplicate; no second event). Twenty identical submissions fired simultaneously yield exactly one row, one 201, and one background job (verified by test).

Device authentication

POST /devices/register stores the device and returns an API key (HMAC-hashed at rest). Submissions require X-Device-Key : missing/invalid → 401; valid key but wrong patient/device → 403.

6 · Background mechanism & reliability

The pipeline is **outbox** → **relay** → **RabbitMQ** → **async consumer**. The sequence below traces one ingest that crosses a clinical threshold all the way to a persisted alert.

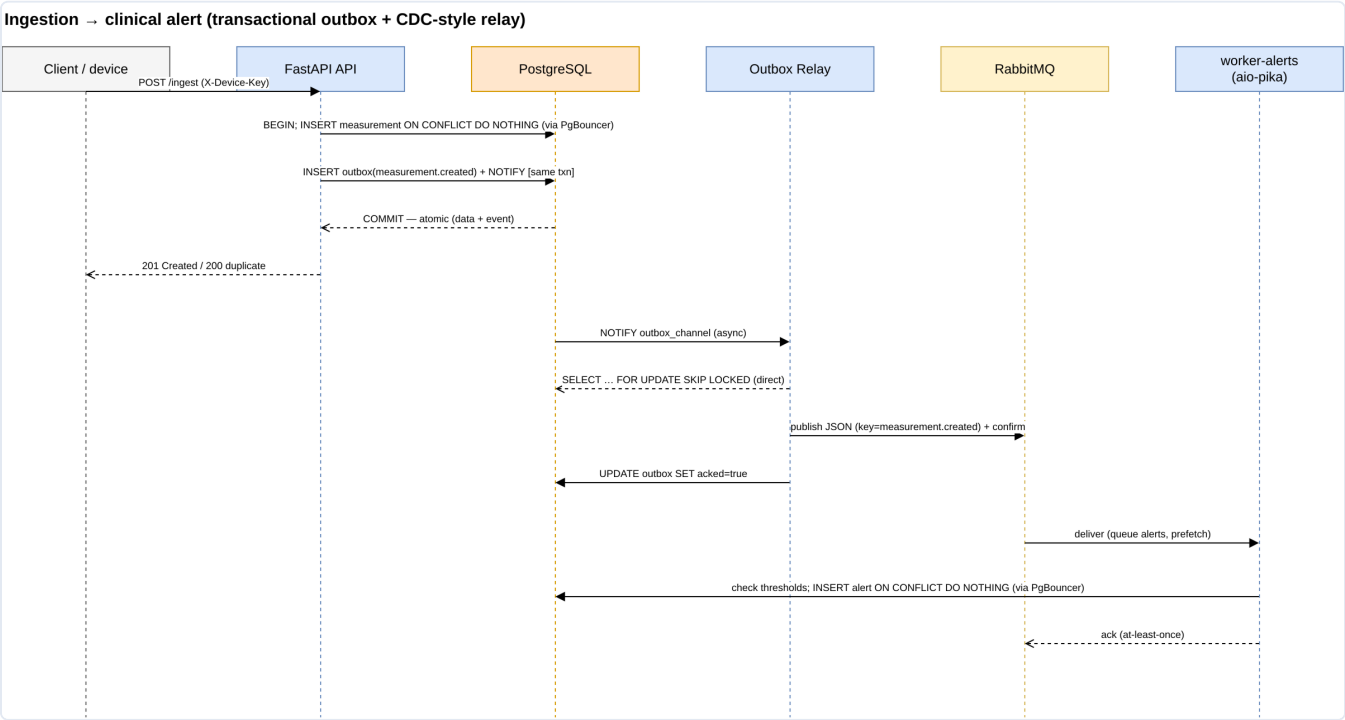


Figure 3 — Ingest → clinical alert. The red boundary in the narrative is the atomic write (measurement + outbox row + NOTIFY); everything after the relay is at-least-once with an exactly-once effect.

Failure scenario	Behaviour	Result
Consumer crash mid-task	message stays unacked → RabbitMQ redelivers	at-least-once; alert created on restart
Relay down	outbox rows stay <code>acked=false</code> ; drained on restart	no loss
RabbitMQ outage	relay cannot publish → rows stay unacked	no loss; drains on recovery
Duplicate / redelivery	UNIQUE constraint blocks the second write	exactly-once effect
Poison message (retries exhausted)	recorded in <code>failed_jobs</code> ; replayable	never silently dropped
Two relay replicas	<code>FOR UPDATE SKIP LOCKED</code>	no double-processing

Proven, not just claimed. A chaos suite kills/stops the consumer, the relay, and the broker, and runs two relays at once — each asserting no loss and no duplicates. Result: **5/5 reliability tests pass**, alongside 27 contract/idempotency/aggregation/risk-score tests (32 passed, 5 chaos opt-in).

7 • Measured performance

Local Docker (8-core laptop), full stack through PgBouncer, single dev API worker (`uvicorn --reload`). Load generator: Locust (`scripts/locustfile.py`), 150 virtual devices, 90 s, ~20% of readings clinically high. To exercise the async pipeline's burst behaviour, `worker-alerts` was stopped during the burst and restarted afterwards.

Metric	Result
Total requests / failures	28,432 / 0 (100% success)
Aggregate throughput	~317 req/s (<code>POST /ingest</code> ~301 req/s)
Ingest latency p50 / p95 / p99	0.13 s / 1.8 s / 3.4 s
Alerts backlog (consumer stopped → restarted)	built to ~18.5k, then drained to 0

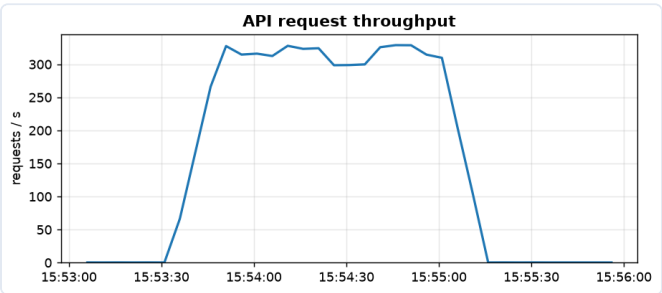


Figure 4 — Request throughput (Prometheus).

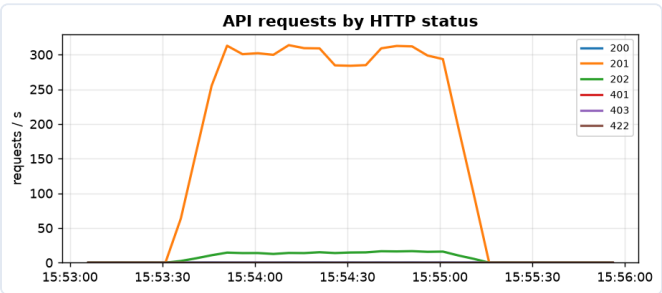


Figure 5 — Requests by HTTP status: all 201/202, no 4xx/5xx.

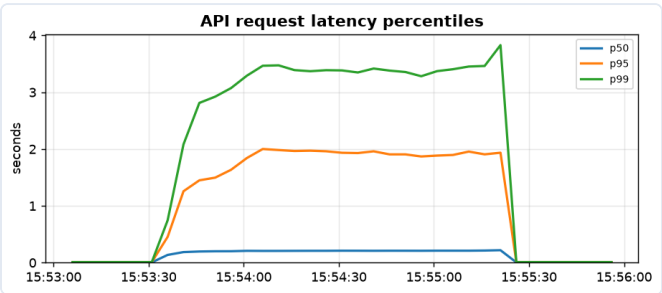


Figure 6 — Latency p50/p95/p99 (high-resolution histogram).

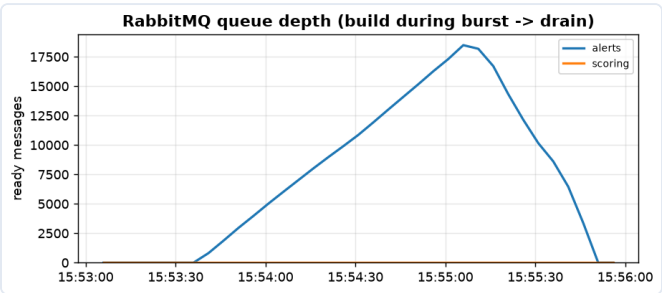


Figure 7 — Alerts queue builds during the burst, then drains to zero — nothing lost.

Reading of it. The system absorbed ~300 req/s with **zero failures** through the pooler. Throughput is bounded by the single dev API worker (one uvicorn process) and the single Postgres — not by the async workers, which drain a ~18.5k backlog cleanly. The queue-depth curve is the async-pipeline guarantee made visible: a burst is buffered durably and worked off, with no data lost.

How to reproduce: `docker compose up --build -d`, then `python -m locust -f scripts/locustfile.py --host http://localhost:8000 --users 150 --run-time 90s --headless --csv out ; Grafana dashboard at http://localhost:3000`, or regenerate the figures with `python scripts/render_metrics.py`.

8 · Production evolution & honest limits

The relay → true CDC

The poll/NOTIFY relay is deliberately lightweight. The natural next step is **change-data-capture** that tails the write-ahead log instead of polling: **wal2json** (a DIY logical-decoding consumer), or a hardened product — **Sequin** or **Debezium**. Because the consumers already read plain JSON, this is a drop-in producer swap. CDC also eases the connection pressure noted in §3: it replaces the relay's regular backend and its polling queries with a **passive replication connection** (a walsender, governed by `max_wal_senders` — separate from the client pool — running no `SELECT/UPDATE` against the table), trading it for the replication-slot concern below.

Recommendation: for production, prefer a **hardened CDC tool (Sequin / Debezium)** over a hand-rolled wal2json consumer. CDC's operational hazard is the **replication slot**: it retains WAL until the consumer confirms, so a lagging/dead consumer can fill the disk; and ordering is subtle (publish + broker-confirm must precede advancing the slot LSN). A managed tool owns exactly this hard, boring, critical code.

The data tier is the ceiling

- **App tier scales horizontally today** (stateless API + workers behind the pooler). The first real ceiling is the single Postgres: connection count (solved by PgBouncer), then write throughput.
- **Cheapest-first levers:** raise API workers/replicas; time-partition `measurements`; batch writes through the queue; then read replicas; a time-series engine (TimescaleDB) for the measurement firehose; sharding only as a last resort.

Known limits (stated honestly)

- RPS here is bounded by one dev uvicorn worker; production runs multiple workers/replicas.
- The outbox relay processes one ordered stream of events (no concurrency to exploit there); it is infra glue, not a request path, so async-first still holds for the API, DB sessions, and consumers.
- One Postgres is the write ceiling; partition/shard is designed but not built.
- Secrets (API-key signing secret, DB password) must move to a secret manager in production.

Appendix · Run it

`docker compose up --build -d` starts ten services — `db`, `pgbouncer`, `rabbitmq`, `one-shot migrate`, `api`, `outbox-relay`, `worker-alerts`, `worker-scoring`, `prometheus`, `grafana`. Schema is applied by the one-shot migrate job (Alembic); the API, relay and workers wait for it. API docs at `/docs`, RabbitMQ UI at `:15672`, Grafana at `:3000`. Tests run against the live stack: `pytest` (add `RUN_CHAOS=1` for the reliability suite).

Generated for the Seerling backend task · async-first solution · all figures produced from the local run described in §7.